

On Tuesday, September 26, 2023, GitLab (git.doit.wisc.edu) will be upgraded from 15.8 to GitLab 16.2. Learn more about the upgrade: <https://kb.wisc.edu/shared-tools/news.php?id=13235>

p

p1

Project ID: 14118



Update P1 due date
ABIGAIL MATTHEWS authored 10 hours ago

Name	Last commit	Last update
input_files	added man_pages and input_files directo...	1 day ago
man_pages	added man_pages and input_files directo...	1 day ago
.gitignore	added .gitignore	1 day ago
README.md	Update P1 due date	10 hours ago

[README.md](#)

P1 - Unix Utilities

Administrivia

- **Due Date:** September 19th, at 11:59pm
- Projects may be turned in up to 3 days late but you will receive a penalty of 10 percentage points for every day it is turned in late.
- **Slip Days:**
 - In case you need extra time on projects, you each will have 2 slip days for individual projects and 2 slip days for group projects (4 total slip days for the semester). After the due date we will make a copy of the handin directory for on time grading.
 - To use a slip days you will submit your files with an additional file **slipdays.txt** in your regular project handin directory. This file should include one thing only and that is a single number, which is the number of slip days you want to use (ie. 1 or 2). Each consecutive day we will make a copy of any directories which contain one of these slipdays.txt files.
 - After using up your slip days you can get up to 90% if turned in 1 day late, 80% for 2 days late, and 70% for 3 days late. After 3 days we won't accept submissions.
 - Any exception will need to be requested from the instructors.
 - Example slipdays.txt

1

- Questions: We will be using Piazza for all questions.
- **Before beginning:** Read this [lab tutorial](#); it has some useful tips for programming in the C environment, including working with makefiles, using the gdb debugger, and using the linux manual and info systems.
- Collaboration: The assignment has to be done by yourself. Copying code (from others) is considered cheating. [Read this](#) for more info on what is OK and what is not. Please help us all have a good semester by not doing this.
- This project is to be done on the [Linux lab machines](#), so you can learn more about programming in C on a typical UNIX-based platform (Linux). Your solution will be tested on these machines.
- Some tests will be provided at `~cs537-1/tests/P1`. Read more about the tests, including how to run them, by executing the command `cat ~cs537-1/tests/P1/README.txt` on any lab machine. Note these test cases are not complete, and you are encouraged to create more on your own.
- **Handing it in:** Copy your files to `~cs537-1/handin/login/P1` where login is your CS login.

Unix Utilities

In this project, you'll build simplified versions of three different UNIX utilities -- **man**, **apropos**, and **groff**. We'll call each of them a slightly different name to avoid confusion with the actual Unix utilities; for example, instead of **man**, you'll be implementing **wman** (i.e., "wisconsin" man).

Objectives:

- Re-familiarize yourself with the C programming language, especially:
 - Working with strings
 - Reading and Writing files
 - Working with structs
- Re-familiarize yourself with the shell / terminal / command-line of UNIX
- Learn a little about how UNIX utilities are implemented

While the project focuses upon writing simple C programs, you can see from the above that even that requires a bunch of other previous knowledge, including a basic idea of what a shell is and how to use the command line on some UNIX-based systems (e.g., Linux or macOS), and of course a basic understanding of C programming. If you **do not** have these skills already, this is not the right place to start.

Summary of what gets turned in:

- Three **.c** files, one for each utility : **wman.c**, **wapropos.c**, and **wgroff.c**.
- Each file should compile successfully when compiled with the `-Wall` and `-Werror` flags.
- They should (hopefully) pass the tests we supply.
- Include a single **README.md** describing the implementation. This file should include your name, your cs login, you wisc ID and email, and the status of your implementation. If it all works then just say that. If there are things you know don't work let me know.
- A **document describing online resources used**. You are allowed to use Large-Language Models and other sources of online inspiration (e.g. stackoverflow). These online resources and generative tools are transforming many industries including computer science and education. *However*, if you use online sources, you are required to turn in a document of all uses of these sources. Indicate in this document what percentage of your solution was done strictly by you and what was done utilizing these tools. Be specific, indicating sources used and how you interacted with those sources. Not giving credit to outside sources is a form of plagiarism. You will not be penalized for using LLMs or reading posts but you should not create posts in online forums about the projects in the course. Be aware that when you seek help from the instructional staff, we will not assist with working with these LLMs and we will expect you to be able to walk the instructional staff member through your code and logic.

wman

The program **wman** is a simplified implementation of the **man** Unix utility. Generally, it searches a directory (in our case the `./man_pages` directory) and its sub-directories (man1 through man9) to find the proper manual page (a file) and then prints its contents. The program can be run with a single command-line argument or two command-line arguments. With a single argument, all sub-directories are searched. With two command-line arguments, only the designated section of the manual should be searched.

A typical single-argument usage is as follows, in which the user wants to see the manual page for the wman command itself:

```
prompt> ./wman wman
wman(1)                                     wman(1)

NAME
    wman - a simplified manual program

SYNOPSIS
    wman [section] page

DESCRIPTION
    wman is a simplified version of man, the system manual program.
    It takes as input the manual page which should be displayed (and an
    optional section number between 1 and 9).  It finds the appropriate
    ...
```

As shown, **wman** finds the manual page for the wman command (wman.1 in the `./man_pages/man1` directory), reads the file, and prints out its contents. The `./` before the **wman** above is a UNIX thing; it just tells the system which directory to find the **wman** program in (in this case, in the `."` (dot) directory, which means the current working directory).

A typical two-argument usage is as follows, in which the user wants to search only within section 2 of the manual for the wman command:

```
prompt> ./wman 2 wman
```

No manual entry for wman in section 2

In this version, only the `./man_pages/man2` sub-directory is searched. Since there is no file called `wman.2` in this directory, the program prints out the message that no `wman` manual page was found within this section.

To create the **wman** binary, you'll be creating a single source file, **wman.c**, and writing C code to implement this version of **man**. To compile this program, you will do the following:

```
prompt> gcc -o wman wman.c -Wall -Werror
prompt>
```

This will make a single *executable binary* called **wman** which you can then run as above.

You'll need to learn how to use a few library routines from the C standard library (often called **libc**) to implement the source code for this program. All C code is automatically linked with the C library, which is full of useful functions you can call to implement your program. Learn more about the C library [here](#) and perhaps [here](#).

For this project, we recommend using the following routines to do file input and output: **fopen()**, **fgets()**, and **fclose()**. Whenever you use a new function like this, the first thing you should do is read about it -- how else will you learn to use it properly?

On UNIX systems, the best way to read about such functions is to use what are called the **man** pages (short for **manual**). In our HTML/web-driven world, the man pages feel a bit antiquated, but they are useful and informative and generally quite easy to use.

To access the man page for **fopen()**, for example, just type the following at your UNIX shell prompt:

```
prompt> man fopen
```

Then, read! Reading man pages effectively takes practice; why not start learning now? To learn about the manual you can use the manual by typing `man man`.

We will also give a simple overview here. The **fopen()** function "opens" a file, which is a common way in UNIX systems to begin the process of file access. In this case, opening a file just gives you back a pointer to a structure of type **FILE**, which can then be passed to other routines to read, write, etc.

Here is a typical usage of **fopen()**:

```
FILE *fp = fopen("wman.1", "r");
if (fp == NULL) {
    printf("cannot open file\n");
    exit(1);
}
```

A couple of points here. First, note that **fopen()** takes two arguments: the *name* of the file and the *mode*. The latter just indicates what we plan to do with the file. In this case, because we wish to read the file, we pass "r" as the second argument. Read the man pages to see what other options are available.

Second, note the *critical* checking of whether the **fopen()** actually succeeded. This is not Java where an exception will be thrown when things go wrong; rather, it is C, and it is expected (in good programs, i.e., the only kind you'd want to write) that you always will check if the call succeeded. Reading the man page tells you the details of what is returned when an error is encountered; in this case, the man page says:

```
Upon successful completion fopen(), fdopen(), and freopen() return a
FILE pointer. Otherwise, NULL is returned and errno is set to indicate
the error.
```

Thus, as the code above does, please check that **fopen()** does not return NULL before trying to use the FILE pointer it returns.

Third, note that when the error case occurs, the program prints a message and then exits with error status of 1. In UNIX systems, it is traditional to return 0 upon success, and non-zero upon failure. Here, we will use 1 to indicate failure.

Side note: if **fopen()** does fail, there are many reasons possible as to why. You can use the functions **perror()** or **strerror()** to print out more about *why* the error occurred; learn about those on your own (using ... you guessed it ... the man pages!).

Once a file is open, there are many different ways to read from it. The one we're suggesting here to you is **fgets()**, which is used to get input from files, one line at a time.

To print out file contents, just use **printf()**. For example, after reading in a line with **fgets()** into a variable **buffer**, you can just print out the buffer as follows:

```
printf("%s", buffer);
```

Note that you should *not* add a newline (`\n`) character to the `printf()`, because that would be changing the output of the file to have extra newlines. Just print the exact contents of the read-in buffer (which, of course, may include a newline).

Finally, when you are done reading and printing, use **`fclose()`** to close the file (thus indicating you no longer need to read from it).

wman Additional Details

Your program **wman** can be invoked with a few options. Here are the cases that you need to correctly handle.

- `wman <page> --` Your program **wman** should search the `./man_pages` sub-directories *man1* through *man9* in order to find a file named `<page>.X` (where `X` is a single digit that is the same as the sub-directory being searched). If found the contents of the file should be printed. If no such file can be found the program should print out exactly "No manual entry for *page*\n" where *page* is the page argument entered on the command line.
- In all non-error cases, **wman** should exit with status code 0, usually by returning a 0 from **main()** (or by calling **exit(0)**).
- If *no page* is specified on the command line, **wman** should print "What manual page do you want?\nFor example, try 'wman wman'\n" and exit and return 0.
- If the program tries to **fopen()** a file and fails, it should print the exact message "cannot open file\n" and exit with status code 1.
- `wman <section> <page> --` Your program **wman** should search the `./man_pages/man` directory to find a file named `.` (where *section* is a decimal number from 1 to 9.) If *section* is not a decimal number in the proper range then print "invalid section\n" and exit with status code of 1. If no such file can be found print "No manual entry for *page* in section *section*\n" where *page* and *section* are the argument entries on the command line.

wapropos

The **apropos** command in the Linux environment searches the manual pages for a keyword and reports a list of all manual pages that use that keyword in the NAME or DESCRIPTION portion of the manual page.

Your version of this command, **wapropos** should search the `./man_pages` sub-directories *man1* through *man9* in order, scanning the NAME and DESCRIPTION portion of every regular, readable file for the keyword given as a command-line argument. The output from the program should be a list of all manual pages containing the keyword in those portions of the file.

A typical usage is as follows, in which the user wants to see a listing of all manual pages containing the keyword *example*:

```
prompt>./wapropos example
example (1) - an example program
wman (1) - a simplified manual program
wapropos (1) - a simplified program to search manual page names and descriptions
example (2) - an example program
```

The keyword must be a single word (i.e. no white space). Every line of the output list should have the form:

```
<page> ( <section> ) - <name_one_liner>
```

Where `page` is the name of the manual page (i.e. the name of the file without the file extension), `<section>` is the section of the manual page (i.e. the file extension), and `<name_one_liner>` is the one-line content of the NAME portion of the manual page after the dash.

wapropos Additional Details

Here are the cases that you **wapropos** program needs to correctly handle.

- `wapropos keyword --` Your program **wapropos** should search the `./man_pages` sub-directories *man1* through *man9* in order to find files named `page.X` (where `X` is a single digit that is the same as the sub-directory being searched). If a file containing `keyword` in the NAME or DESCRIPTION portion of the file is found it should be included in the output list. The output list must be in the form: `<page> ( <section> ) - <name_one_liner>` where `<page>` is the manual page name (i.e. the filename without the extension), `<section>` is the section of the manual page (i.e. the file extension), and `<name_one_liner>` is the one-line content of the NAME portion of the manual page after the dash. The program should exit with code 0.
- If no manual pages are found containing the keyword then the program should print "nothing appropriate\n" and exit with code 0.
- If no keyword is provided on the command line then the program should print "wapropos what?\n" and exit with code 0.

wgroff

The **groff** program is a document formatting system that can convert input files to many different types of output formats including properly formatted manual pages. To create manual page, input files must follow a very specific format.

Your **wgroff** program will handle a subset of format conversion for manual pages. The input text file must follow the following rules and perform the following conversion from input to output:

- Lines that begin with a hashtag (`#`) are ignored.

- The first line must begin with a `.TH` (title header) and must be of the form: `.TH <command> <section> <date>` where `<command>` is the program or manual page that is being created, the `<section>` is a number between 1 and 9 for the section the page should be placed in, and `<date>` is a date in the form YYYY-MM-DD. This line controls the name of the output file which should be `<command>.<section>` and placed in the current directory. This line controls the content of the first and last line of the output file. The first and last line must be exactly 80 characters long. The first line should have the form `<command> (<section>)` appear twice -- one left justified and one right justified with spaces between to be exactly 80 characters wide. The last line should also be exactly 80 characters wide and have `<date>` centered on the line using spaces for padding.
- Lines that begin with `.SH` are section headers and are of the form: `.SH <section name>`. The `<section name>` can contain any characters (except a newline). The output file should contain `<section name>` but converted to upper-case and should be in bold (see below). A blank line should be added to the output file before every section header.
- All other lines of the input file should be write to the output file after scanning and converting the following formatting marks on the line and indenting the content 7 spaces:
 - `/fB` should be converted to `/033[1m` (ANSI bold)
 - `/fI` should be converted to `/033[3m` (ANSI italic)
 - `/fU` should be converted to `/033[4m` (ANSI underline)
 - `/fP` should be converted to `/033[0m` (ANSI reset to normal)
 - `//` should be converted to a `/` (output forward-slash)

wgroff Additional Details

Here are the cases that you **wgroff** program needs to correctly handle.

- `wgroff input_file --` Your program should take one command line argument, the name of a properly formatted input file (see above for proper formatting). It should create an output file and write to that output file the converted version of the content from the input file. See above for how the input should be converted. After the program successfully completes the conversion and closes the output file it should exit with code 0.
- If no input file is provided on the command line the program should print "Improper number of arguments\nUsage: ./wgroff <file> \n" and exit with code 0.
- If input file provided on the command line doesn't exist, the program should print "File doesn't exist\n" and exit with code 0.
- If the input file is improperly formatted, the program should output "Improper formatting on line <lineno> \n" and exit with code 0. The `<lineno>` should be the first line in the input file that is improperly formatted.